🚀 Lecture 02.08: Assignment Operators & In-Place Mutation

📌 Executive Architectural Summary

Assignment operators bind values to identifier names. Compound (augmented) assignment operators combine a binary arithmetic operation with re-assignment, optimizing in-place mutation where supported by the underlying C type.

🔬 Core Engineering Mechanics

1. Augmented Operators Mapping

- `x += c` $\equiv x = x + c$ (calls `__iadd__`, falling back to `__add__`)
- `x -= c` $\equiv x = x - c$ (calls `__isub__`)
- `x *= c` $\equiv x = ximesc$ (calls `__imul__`)
- `x /= c` $\equiv x = x/c$ (calls `__itruediv__`)
- `x //= c` $\equiv x = \lfloor x/cfloor$ (calls `__ifloordiv__`)
- `x %= c` $\equiv x = x\%c$ (calls `__imod__`)
- `x **= c` $\equiv x = x^c$ (calls `__ipow__`)

2. Immutability Implication

For immutable types (`int`, `float`, `str`, `tuple`), augmented assignment **always creates a new object** and rebinds the identifier to the new memory address.

💻 Production Python Implementation

```
x = 10
print(f"Initial: x = {x} (id: {id(x)})")

x += 5
print(f"After x += 5: x = {x} (id changed: {id(x)})")

x *= 2
print(f"After x *= 2: x = {x}")

x /= 4
```

```
print(f"After x /= 4: x = {x} (converted to float)")

x **= 3
print(f"After x **= 3: x = {x}")
```